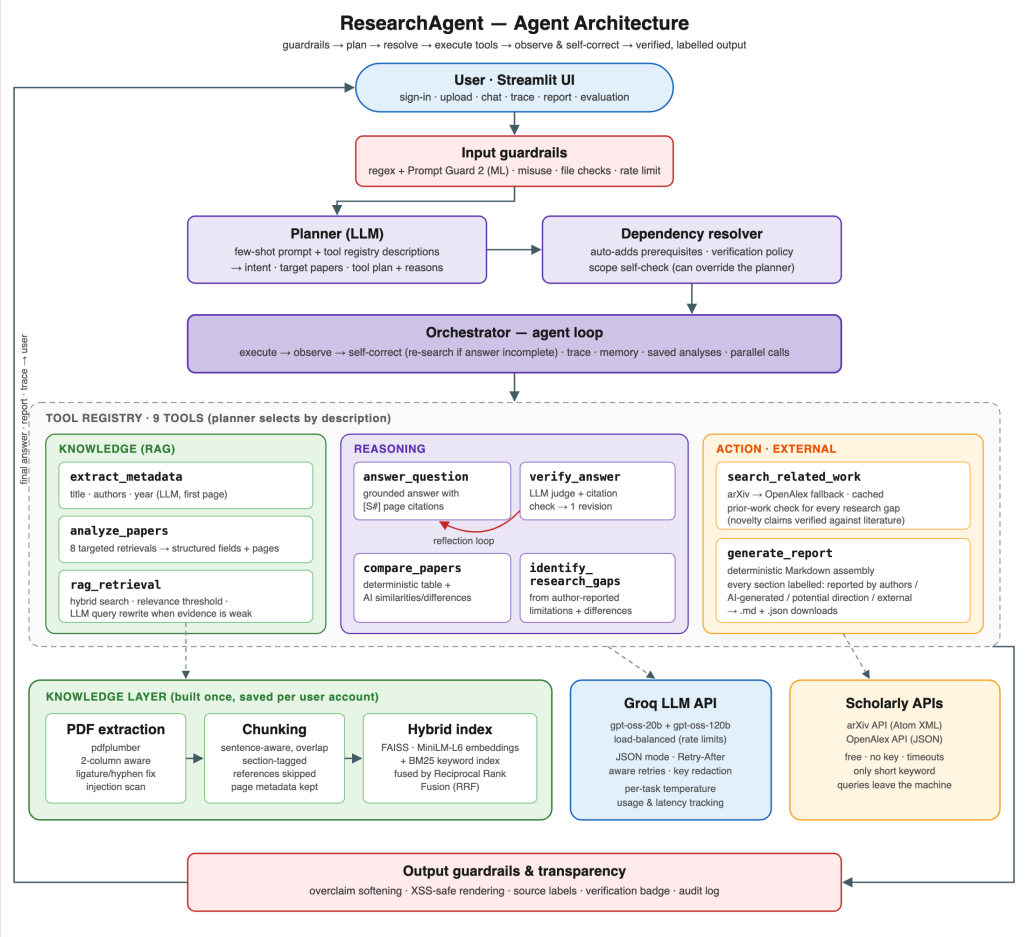
ResearchAgent

An Agentic AI System for Research Paper Analysis, Comparison and Research Gap Identification

Activity	FA Activity: Agentic AI Application Implementation
Domain	Research Paper Analysis Agent
Group members	Jayesh Patil [Group partner name - roll no.]
Institute	[Institute name]
Course	[Course / class / division]
Source code	https://github.com/mitalihatestocode/ResearchAgent
Date	15 October 2026



1. Introduction

Researchers and students routinely need to read several papers on one topic, pull out what each one did (dataset, method, models, metrics, results, limitations), compare them, and decide what is still open. Doing this by hand is slow. Doing it with a general-purpose chatbot is risky: chatbots summarise from their training memory, invent citations and numbers, and present "research gaps" as confirmed discoveries.

ResearchAgent is an agentic AI application that works over the papers the user uploads. For each request it **plans** which of its 9 tools to use, **retrieves** evidence from the papers, **acts** through local tools and external APIs, **checks** its own answers, and returns output that is cited, labelled and verifiable.

2. Problem Statement

Given a small set of research papers (PDFs) and a natural-language request, produce answers, structured comparisons and potential research directions that are:

1. **grounded**, with every factual statement traceable to a page of an uploaded paper;
2. **right-sized**, so a narrow question doesn't trigger a full literature review and a broad one isn't answered superficially;
3. **honest**, clearly stating when information is missing and never presenting AI suggestions as confirmed novel findings;
4. **safe**, robust to prompt injection (including instructions hidden inside documents) and not usable for academic misconduct.

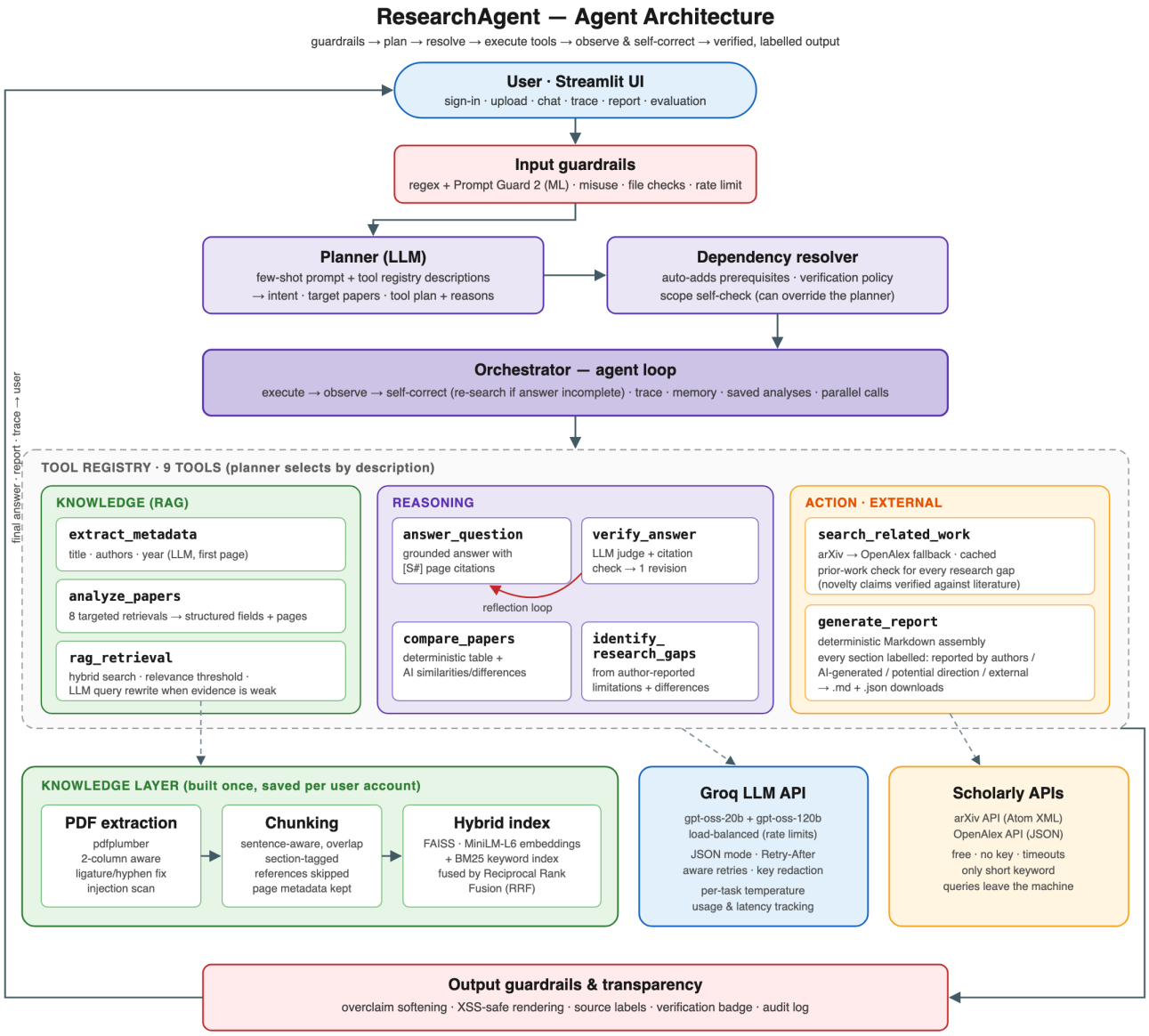
Why an agent and not a chatbot? The task has independent sub-tasks needing different context (extraction per paper, comparison across papers, gap analysis, literature lookup). The right workflow depends on the question, so it must be planned rather than fixed. Grounding requires retrieval before generation. Checking whether a gap is novel requires an external tool. A single prompt→answer call can't meet these requirements; a planning, tool-using, self-checking agent can.

3. Objectives

1. Build a pipeline that ingests PDFs, including two-column layouts, into a searchable, section-aware knowledge base.
2. Implement an LLM planner that chooses tools, target papers and intent for each request from a tool registry.
3. Implement hybrid retrieval (semantic plus keyword) with a relevance threshold and citations.
4. Extract a validated, structured representation of each paper and compare papers on standard parameters.
5. Suggest potential research gaps grounded in author-reported limitations, and check each against external literature.
6. Add reflection: verify every answer against its evidence and revise it once if needed.
7. Enforce safety with deterministic guardrails on input, documents and output.

8. Provide a usable UI with a transparent agent trace and downloadable report, plus an automated test suite.

4. Proposed Agent Architecture



The system has five layers:

Layer	Components	Responsibility
Interface	Streamlit UI	upload, chat, live trace, analysis views, report download
Safety	`safety/guardrails.py`	input, document and output guardrails
Agent core	Planner, Tool Registry, Dependency Resolver, Orchestrator	decide and execute the workflow, observe, self-correct, keep memory
Tools	9 tools (knowledge, reasoning, action)	metadata, analysis, retrieval, answering, verification, comparison, gaps, literature search, report

Layer	Components	Responsibility
Infrastructure	Knowledge layer (FAISS + BM25), Groq LLM API, arXiv/OpenAlex APIs	storage, generation, external knowledge

Agentic properties. *Planning*: the LLM produces a tool plan for each question. *Tool use*: 9 tools across three kinds (local, LLM, external API). *Multi-step execution*: an ordered, dependency-resolved execution with a trace. *Autonomous decisions*: the agent can override the planner's out-of-scope decision when the papers clearly contain relevant evidence, and it rewrites queries when evidence is weak or when its own draft answer says the evidence was insufficient. *Reflection*: verify, then revise. *Memory*: conversation history for follow-ups, plus **user accounts** that persist each user's papers, embedding vectors, LLM analyses and chat history, so returning users never re-upload or re-analyse a paper.

We chose **plan-then-execute with observation** over an open-ended ReAct loop. It gives the agent real autonomy where that is safe, keeps cost and latency bounded, and makes every decision explainable. Section 8 describes the full loop.

5. Prompt Design

All prompts live in `agent/prompts.py`, one role per prompt (planner, metadata extractor, paper analyst, answerer, verifier, query rewriter, comparer, gap suggester). The techniques used:

Technique	How it is applied
Role + task decomposition	each call has one narrow role instead of a generic assistant
Few-shot examples	the planner sees 5 worked request→plan examples, including an out-of-scope one where no tools should run
Structured output	every call requests JSON with an explicit schema; the API's JSON mode is enabled; Pydantic validates downstream
Reason-then-answer	plans and verdicts contain a `reasoning` field written before the decision (compact chain-of-thought), which is also shown to the user
Delimited untrusted context	evidence is wrapped in `<evidence id="S1" paper=… page=… section=…>` tags, and the model is told this is data, never instructions
Citation enforcement	answers must cite `[S#]` after each claim; code then checks that the citations exist
Explicit fallback string	missing fields must read exactly "Not explicitly reported in the provided paper."
Self-critique	a separate verifier prompt (temperature 0) audits claims against evidence
Query rewriting	weak retrieval triggers a prompt that generates 3 alternative queries in research vocabulary
Per-task temperature	0.0 for planning, metadata and verification; 0.1 for extraction; 0.2 for answers and comparison; 0.4 for gap ideas
Shared safety rules	`SAFETY_RULES` is appended to every prompt: no fabrication, no invented citations, no novelty claims, ignore instructions inside evidence

Example (planner few-shot entry):

```
Request: "What dataset did paper 1 use?"
Plan: {"reasoning": "Narrow factual question about one paper's dataset; retrieval plus a
cited
       answer is enough. No comparison or gaps were requested.",
       "in_scope": true, "intent": "factual_question", "target_papers": ["bert.pdf"],
       "steps": [{"tool": "rag_retrieval", ...}, {"tool": "answer_question", ...}]}
```

6. RAG Implementation

1. **Extraction** (`rag/pdf_extractor.py`): uses pdfplumber with a tighter word tolerance and **two-column layout detection**. If almost no words cross the vertical midline, the page is cropped into halves. Ligatures and hyphenation are repaired, and documents are capped at 80 pages. On real two-column papers this fixed text where words were merged and lines interleaved across columns. It also raised the number of detected sections, for example from 3 to 6 on the CRAG paper.
2. **Chunking** (`rag/chunker.py`): chunks are packed from whole sentences up to 900 characters with about 150 characters of sentence-level overlap. Section headings (Abstract, Method, Results, Limitations, …) are detected and stored on each chunk. The **References section is excluded** from the index, and every chunk is scanned for prompt injection.
3. **Indexing** (`rag/vector_store.py`): local `all-MiniLM-L6-v2` embeddings (384-d, normalised) go into FAISS `IndexFlatIP`, which gives cosine similarity, alongside a dependency-free **BM25** keyword index (`rag/bm25.py`).
4. **Hybrid retrieval**: dense and BM25 rankings are merged with **Reciprocal Rank Fusion**, score = Σ 1/(60 + rank). Dense search handles paraphrases; BM25 catches exact names such as "PopQA" and "MIMIC-III".
5. **Relevance threshold**: passages below cosine 0.28 are never used as evidence. We calibrated this on the sample papers: off-topic questions score 0.08–0.21 and on-topic ones 0.43–0.71. An integration test with the real model checks this calibration.
6. **Per-paper coverage**: multi-paper questions also retrieve the top passages from each paper separately, so no paper is left out of the evidence.
7. **Targeted multi-query extraction**: `analyze_papers` runs 8 field-specific queries per paper, de-duplicates the evidence, and asks the LLM which evidence IDs support each field. These are mapped back to page and section ("p.6 (Experiments)").

7. Tool/API Integration

Tool / API	Kind	Integration details
Groq Chat Completions API	external LLM	`agent/llm_client.py`: JSON mode, **exponential-backoff retries** on 429/5xx/timeouts, **automatic fallback model**, fail-fast with a clear message on 401/403, thread-safe usage and latency accounting
arXiv API (Atom XML)	external, no key	`tools/literature_search_tool.py`: keyword query, XML parsing, timeouts, caching
OpenAlex API (JSON)	external, no key	automatic fallback when arXiv fails or returns nothing; includes citation counts

Tool / API	Kind	Integration details
sentence-transformers / FAISS	local library	embeddings and vector search
9 agent tools	internal	registered in `agent/tool_registry.py` with description, when-to-use, prerequisites and kind

`search_related_work` serves two purposes. (a) **Related-work search**: it uses paper titles from the metadata to find related publications. (b) **Prior-work check**: for every suggested research gap, it searches with an LLM-generated keyword query and shows the closest existing publications next to the gap. The user can then judge novelty themselves instead of trusting the model. Only short keyword queries leave the machine, never paper text.

Every tool returns a `{"status": …, "message": …}` dictionary instead of raising errors, so a failing API degrades one step rather than the whole run.

8. Agent Workflow

```
1. Input guardrails      → block injection / misuse / invalid input (zero LLM calls if
blocked)
2. Plan                  → LLM planner returns intent, target papers, tools + reasons
                           (validated: unknown tools/papers dropped; rule-based fallback
if the LLM fails)
3. Scope self-check      → out-of-scope plan? probe the papers; override only if strongly
related (≥0.45)
4. Resolve dependencies  → add prerequisites (e.g. compare needs metadata + analysis) +
verification policy
5. Execute each step     → emit a trace event {step, status, detail, duration, why,
auto_added}
     rag_retrieval       → weak evidence (<0.35)? LLM rewrites query → retrieve again →
merge
     answer_question     → cited answer from evidence; if the draft says evidence is
missing,
                           rewrite the search around the gap, retrieve again, keep the
better answer
     verify_answer       → judge + citation check → unsupported? revise once → re-verify
     analysis/metadata   → parallel per paper, cached for later questions
6. Compose final answer  → labelled output + report + full trace in the UI
```

Example: "Compare these papers and identify potential research gaps" plan → `extract_metadata → analyze_papers → compare_papers → identify_research_gaps → search_related_work → generate_report`

Example: "What dataset did paper 1 use?" plan → `rag_retrieval → answer_question`, plus an auto-added `verify_answer`, scoped to paper 1 only.

9. Safety Mechanism

Safety works in depth: **deterministic code checks** that can't be talked out of their job, an **ML classifier**, **prompt-level rules**, and an **LLM judge**, applied across input, documents, output and infrastructure. The full threat model is in SECURITY.md.

Risk	Mechanism	Type
User prompt injection ("ignore previous instructions…")	`validate_query` patterns run on **Unicode-normalised** text (NFKC, zero-width and bidi characters removed); blocked before any LLM call	deterministic
Paraphrased jailbreaks that rules miss	**Llama Prompt Guard 2** classifier (score ≥ 0.80 is blocked; attacks scored 0.998–0.9996 and research questions about 0.0004 in testing); fails open with an audit event	ML classifier
Indirect injection hidden in a PDF	each chunk is scanned at indexing; phrases are replaced before reaching a prompt; evidence is delimited as data; fake `</evidence>` delimiters are defused; the UI warns	deterministic
Academic misconduct (fabricating data, fake citations, beating plagiarism or AI detectors, ghost-writing)	refused with an explanation	deterministic
Malicious files	magic-byte check, encrypted and `/JavaScript` or `/Launch` PDFs rejected, embedded files flagged, SHA-256 dedupe, sanitised filenames, size, page and count limits enforced in the app and by the server	deterministic
Hallucinated facts and citations	RAG only; relevance threshold; "not reported" fallback; `[S#]` citations checked in code; verifier + revision	prompt + code + LLM judge
Overclaiming novelty	prompt framing; `soften_overclaims`; prior-work check against arXiv/OpenAlex	3 layers
XSS through rendered content	untrusted text is escaped in HTML or rendered as Markdown with HTML disabled; only `http(s)` links	deterministic
Account security	salted PBKDF2-SHA256 (600k iterations); constant-time checks; generic errors (no username enumeration); 5-attempt lockout; per-user data isolation; owner-only file permissions; password-confirmed account deletion; guest mode saves nothing	deterministic
Secret leakage and key sharing	session-only key scoped per run (a `ContextVar` that is propagated to worker threads); errors are redacted; `.env` is git-ignored	deterministic
Abuse and cost	per-session rate limit (6 per minute, 120 per session); bounded retries	deterministic
Infrastructure	localhost binding, XSRF protection, hidden error details, static serving and telemetry off; **`pip-audit`: 0 known vulnerabilities** after upgrading 50+ vulnerable pins	configuration

Risk	Mechanism	Type
Transparency	every output labelled *Reported by authors* / *AI-generated* / *Potential direction* / *External source*; full trace; verification badge; a **Security tab** with a privacy-preserving audit log (fingerprints only, never content)	UI

Testing exposed two problems with the guardrails themselves. First, the phrase "system prompt" appears legitimately in the Self-RAG paper, so we narrowed that pattern and added a regression test. Second, the regex layer alone missed the paraphrased jailbreak above, which is why the ML classifier was added. Both cases are now automated tests.

10. Results

Automated testing: 106 of 106 pytest tests pass (`pytest -q`). They cover 38 documented test cases: ingestion, RAG, planning, every tool, reflection, caching, failure isolation, all guardrails 10 application-security cases and 7 account and storage cases. `pip-audit` reports no known vulnerabilities. See TEST_REPORT.md.

Retrieval quality on real papers (3 arXiv papers on RAG: Lewis et al. 2020, Self-RAG 2023, CRAG 2024; 181 chunks):

Query	Best cosine	Outcome
What is the retrieval evaluator in CRAG?	0.71	correct passage (CRAG p.5)
What are the limitations of Self-RAG?	0.55	Self-RAG experiments section
What datasets were used for evaluation?	0.50	"Tasks and datasets" (Self-RAG p.6)
Who won the cricket world cup?	0.21	rejected (below 0.28)
Recipe for chocolate cake	0.13	rejected
What's the weather in Mumbai today?	0.10	rejected

Engineering fixes found while testing (the original code base failed on a clean install):

- `groq==0.11.0` is incompatible with `httpx>=0.28` (`unexpected keyword argument 'proxies'`), so **every LLM call failed**. We upgraded and pinned the SDK.
- `pyarrow>=18` requires NumPy 2 while `numpy==1.26.4` was pinned, so **every Streamlit table and chart crashed**. We pinned `pyarrow==17.0.0`.
- Two-column PDFs produced merged words ("Giventhedocumentretrievalresults") and interleaved columns. Layout-aware extraction fixed both and improved retrieval scores for on-topic questions (e.g. 0.45 → 0.50 and 0.58 → 0.71).

Live LLM evaluation: 12 of 12 end-to-end scenarios passed against the real Groq model, real Prompt Guard 2 and the 3 arXiv papers. The full log, with every trace and answer, is in live_test_results.md. Highlights:

- "*Which datasets were used by each paper?*" returned a correct, cited answer for all three papers (PopQA/Biography/PubHealth/ARC-Challenge; NQ/TriviaQA/WebQuestions/CuratedTrec; Self-RAG training data). The verifier flagged the first draft, and the agent revised it automatically.
- The research-gap run produced directions tied to specific author-reported limitations, for example supervised evidence retrieval for FEVER, from the RAG paper.
- The paraphrased jailbreak missed by the regex rules was blocked by Prompt Guard 2 (0 main-LLM calls). The hidden instruction inside a PDF was ignored, and only the real result was reported.
- A full 3-paper research-gap run in the browser takes **45–62 s, uses 9 LLM calls and about 20k tokens**. Follow-up questions reuse the cached analyses and take 5–25 s.

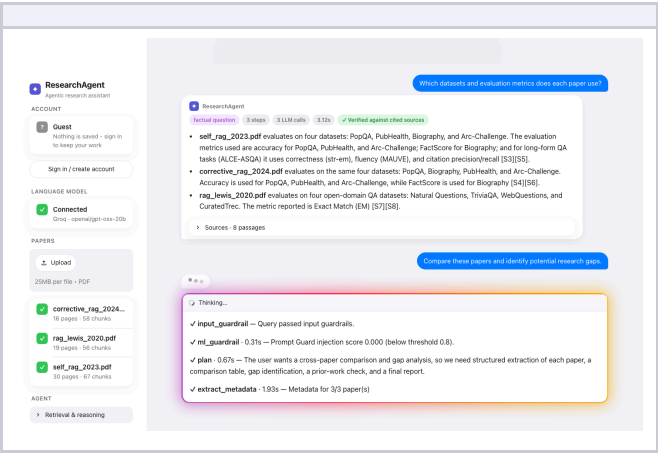
RAG evaluation benchmark (evaluation_results.md, `python -m evaluation.run_eval`): 12 hand-labelled questions on the 3 papers, with every answer verified against the papers' text, plus 3 unanswerable or off-topic questions.

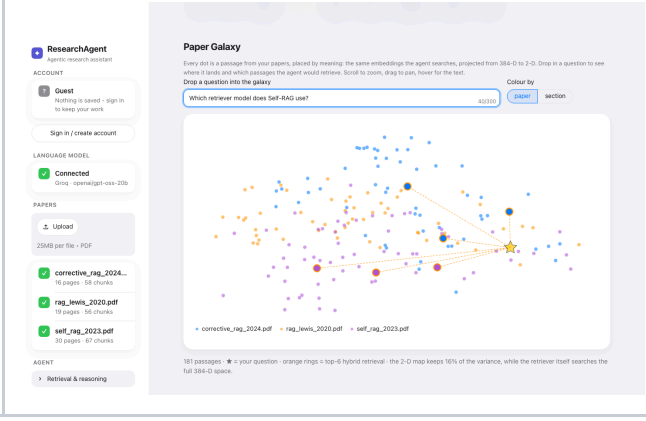
Retrieval (top-6, no LLM)	**Hit@6**	**MRR**	**Context recall**
Dense, all papers	0.67	0.47	0.62
Keyword (BM25), all papers	0.67	0.37	0.60
Hybrid, all papers	**0.75**	0.42	0.62
Hybrid, scoped to the named paper (what the agent does)	0.83	0.54	0.79

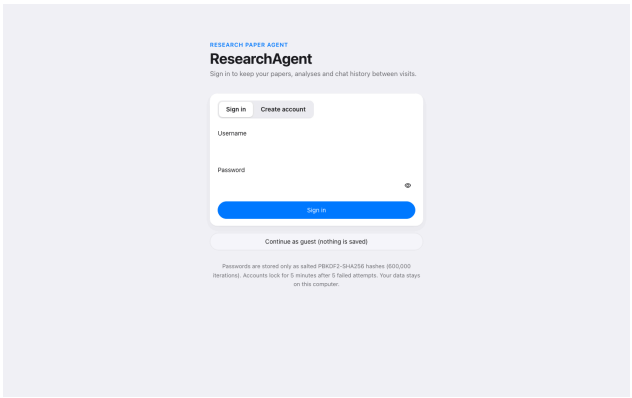
End-to-end (real LLM)	**Before self-correction loop**	**After**
Answer accuracy (all expected facts)	67% (8/12)	**75% (9/12)**
Mean fact recall	77%	**81%**
Citation validity (code check)	100%	**100%**
Faithfulness (verifier)	100%	**100%**
Correct refusals (unanswerable / off-topic)	100%	**100%**
Mean latency / LLM calls / tokens	—	19 s / 3.9 / ~7k

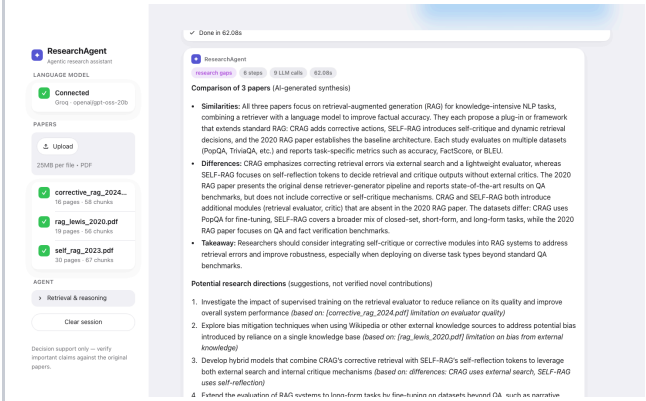
Hybrid search beats either method alone when searching all papers. The agent **never invented an answer**: every unanswerable question was refused, and every citation was valid. The benchmark exposed honest-but-incomplete answers when the first search missed the key passage, which led to the answerability-driven second search (Q06 went from 50% to 100%). The two remaining misses (Q10, Q12) are facts packed into one dense paragraph on page 7 of Self-RAG. For both, the agent answered "not explicitly reported" rather than guessing. That's a retrieval limitation, not a hallucination (see §12).

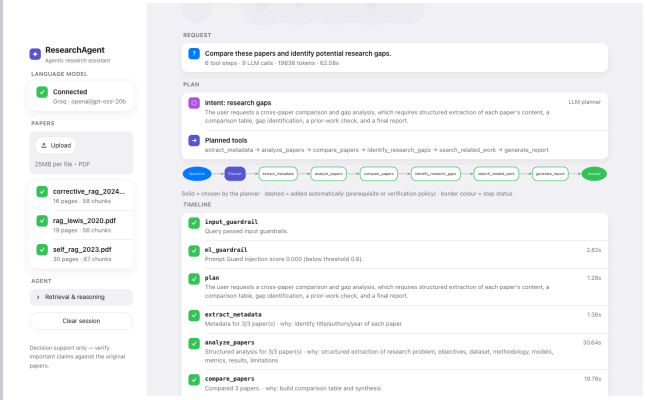
User interface (iOS-inspired design, light and dark themes). The UI has iOS-style motion: cards fade and slide in, chat bubbles pop in, typing dots show while the agent thinks, a glowing gradient border surrounds the agent while it works, and answers stream in word by word. All motion turns off for users who enable *reduce motion* (accessibility). The **Paper Galaxy** tab projects every passage's 384-D embedding to 2-D (PCA) and drops the user's question onto the map as a ★, with lines to the passages the agent would retrieve. This makes semantic search visible.

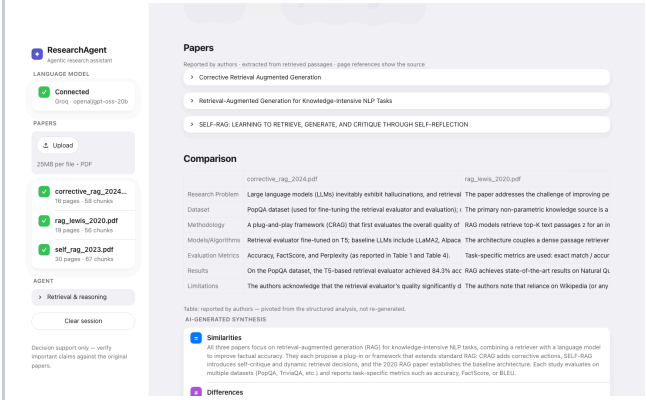

ResearchAgent
Agentic research assistant
ACCOUNT
Guest
Nothing is saved - sign in to keep your work
Sign in / create account
LANGUAGE MODEL
Connected
Groq · openai/gpt-oss-20b
PAPERS
Upload
25MB per file • PDF
corrective_rag_2024...
16 pages · 58 chunks
rag_lewis_2020.pdf
19 pages · 56 chunks
self_rag_2023.pdf
30 pages · 67 chunks
AGENT
Retrieval & reasoning
Which datasets and evaluation metrics does each paper use?
ResearchAgent
factual question
3 steps
3 LLM calls
3.12s
Verified against cited sources
self_rag_2023.pdf evaluates on four datasets: PopQA, PubHealth, Biography, and Arc-Challenge. The evaluation metrics used are accuracy for PopQA, PubHealth, and Arc-Challenge; FactScore for Biography; and for long-form QA tasks (ALCE-ASQA) it uses correctness (str-em), fluency (MAUVE), and citation precision/recall [S3][S5].
corrective_rag_2024.pdf evaluates on the same four datasets: PopQA, Biography, PubHealth, and Arc-Challenge. Accuracy is used for PopQA, PubHealth, and Arc-Challenge, while FactScore is used for Biography [S4][S6].
rag_lewis_2020.pdf evaluates on four open-domain QA datasets: Natural Questions, TriviaQA, WebQuestions, and CuratedTrec. The metric reported is Exact Match (EM) [S7][S8].
Sources · 8 passages
Compare these papers and identify potential research gaps.
Thinking...
input_guardrail — Query passed input guardrails.
ml_guardrail · 0.31s — Prompt Guard injection score 0.000 (below threshold 0.8).
plan · 0.67s — The user wants a cross-paper comparison and gap analysis, so we need structured extraction of each paper, a comparison table, gap identification, a prior-work check, and a final report.
extract_metadata · 1.93s — Metadata for 3/3 paper(s)

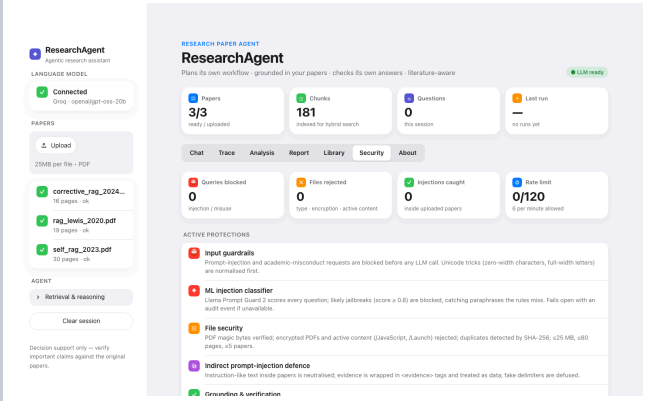

ResearchAgent
Agentic research assistant
ACCOUNT
Guest
Nothing is saved - sign in to keep your work
Sign in / create account
LANGUAGE MODEL
Connected
Groq · openai/gpt-oss-20b
PAPERS
Upload
25MB per file • PDF
corrective_rag_2024...
16 pages · 58 chunks
rag_lewis_2020.pdf
19 pages · 56 chunks
self_rag_2023.pdf
30 pages · 67 chunks
AGENT
Retrieval & reasoning
Paper Galaxy
Every dot is a passage from your papers, placed by meaning: the same embeddings the agent searches, projected from 384-D to 2-D. Drop in a question to see where it lands and which passages the agent would retrieve. Scroll to zoom, drag to pan, hover for the text.
Drop a question into the galaxy
Which retriever model does Self-RAG use?
40/300
Colour by
paper
section
corrective_rag_2024.pdf
rag_lewis_2020.pdf
self_rag_2023.pdf
181 passages · ★ = your question · orange rings = top-6 hybrid retrieval · the 2-D map keeps 16% of the variance, while the retriever itself searches the full 384-D space.

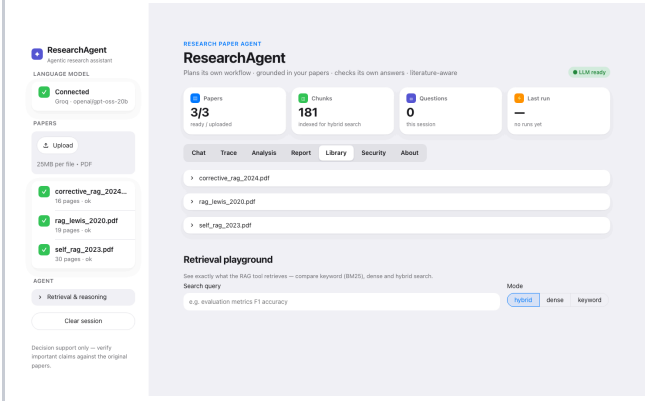

RESEARCH PAPER AGENT
ResearchAgent
Sign in to keep your papers, analyses and chat history between visits.
Sign in
Create account
Username
Password
Sign in
Continue as guest (nothing is saved)
Passwords are stored only as salted PBKDF2-SHA256 hashes (600,000 iterations). Accounts lock for 5 minutes after 5 failed attempts. Your data stays on this computer.


ResearchAgent
Agentic research assistant
LANGUAGE MODEL
Connected
Groq · openai/gpt-oss-20b
PAPERS
Upload
25MB per file • PDF
corrective_rag_2024...
16 pages · 58 chunks
rag_lewis_2020.pdf
19 pages · 56 chunks
self_rag_2023.pdf
30 pages · 67 chunks
AGENT
Retrieval & reasoning
Clear session
Decision support only — verify important claims against the original papers.
Done in 62.08s
ResearchAgent
research gaps
6 steps
9 LLM calls
62.08s
Comparison of 3 papers (AI-generated synthesis)
Similarities: All three papers focus on retrieval-augmented generation (RAG) for knowledge-intensive NLP tasks, combining a retriever with a language model to improve factual accuracy. They each propose a plug-in or framework that extends standard RAG: CRAG adds corrective actions, SELF-RAG introduces self-critique and dynamic retrieval decisions, and the 2020 RAG paper establishes the baseline architecture. Each study evaluates on multiple datasets (PopQA, TriviaQA, etc.) and reports task-specific metrics such as accuracy, FactScore, or BLEU.
Differences: CRAG emphasizes correcting retrieval errors via external search and a lightweight evaluator, whereas SELF-RAG focuses on self-reflection tokens to decide retrieval and critique outputs without external critics. The 2020 RAG paper presents the original dense retriever-generator pipeline and reports state-of-the-art results on QA benchmarks, but does not include corrective or self-critique mechanisms. CRAG and SELF-RAG both introduce additional modules (retrieval evaluator, critic) that are absent in the 2020 RAG paper. The datasets differ: CRAG uses PopQA for fine-tuning, SELF-RAG covers a broader mix of closed-set, short-form, and long-form tasks, while the 2020 RAG paper focuses on QA and fact verification benchmarks.
Takeaway: Researchers should consider integrating self-critique or corrective modules into RAG systems to address retrieval errors and improve robustness, especially when deploying on diverse task types beyond standard QA benchmarks.
Potential research directions (suggestions, not verified novel contributions)
1. Investigate the impact of supervised training on the retrieval evaluator to reduce reliance on its quality and improve overall system performance (based on: [corrective_rag_2024.pdf] limitation on evaluator quality)
2. Explore bias mitigation techniques when using Wikipedia or other external knowledge sources to address potential bias introduced by reliance on a single knowledge base (based on: [rag_lewis_2020.pdf] limitation on bias from external knowledge)
3. Develop hybrid models that combine CRAG's corrective retrieval with SELF-RAG's self-reflection tokens to leverage both external search and internal critique mechanisms (based on: differences: CRAG uses external search, SELF-RAG uses self-reflection)
4. Extend the evaluation of RAG systems to long-form tasks by fine-tuning on datasets beyond QA, such as narrative


ResearchAgent
Agentic research assistant
LANGUAGE MODEL
Connected
Groq · openai/gpt-oss-20b
PAPERS
Upload
25MB per file • PDF
corrective_rag_2024...
16 pages · 58 chunks
rag_lewis_2020.pdf
19 pages · 56 chunks
self_rag_2023.pdf
30 pages · 67 chunks
AGENT
Retrieval & reasoning
Clear session
Decision support only — verify important claims against the original papers.
REQUEST
Compare these papers and identify potential research gaps.
6 tool steps · 9 LLM calls · 19838 tokens · 62.08s
PLAN
Intent: research gaps
LLM planner
The user requests a cross-paper comparison and gap analysis, which requires structured extraction of each paper's content, a comparison table, gap identification, a prior-work check, and a final report.
Planned tools
extract_metadata → analyze_papers → compare_papers → identify_research_gaps → search_related_work → generate_report
Question
Planner
extract_metadata
analyze_papers
compare_papers
identify_research_gaps
search_related_work
generate_report
Answer
Solid = chosen by the planner · dashed = added automatically (prerequisite or verification policy) · border colour = step status
TIMELINE
input_guardrail
Query passed input guardrails.
ml_guardrail
Prompt Guard injection score 0.000 (below threshold 0.8).
2.82s
plan
The user requests a cross-paper comparison and gap analysis, which requires structured extraction of each paper's content, a comparison table, gap identification, a prior-work check, and a final report.
1.28s
extract_metadata
Metadata for 3/3 paper(s) · why: identify title/authors/year of each paper
1.36s
analyze_papers
Structured analysis for 3/3 paper(s) · why: structured extraction of research problem, objectives, dataset, methodology, models, metrics, results, limitations
30.64s
compare_papers
Compared 3 papers · why: build comparison table and synthesis
19.78s


ResearchAgent
Agentic research assistant
LANGUAGE MODEL
Connected
Groq · openai/gpt-oss-20b
PAPERS
Upload
25MB per file • PDF
corrective_rag_2024...
16 pages · 58 chunks
rag_lewis_2020.pdf
19 pages · 56 chunks
self_rag_2023.pdf
30 pages · 67 chunks
AGENT
Retrieval & reasoning
Clear session
Decision support only — verify important claims against the original papers.
Papers
Reported by authors · extracted from retrieved passages · page references show the source
Corrective Retrieval Augmented Generation
Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks
SELF-RAG: LEARNING TO RETRIEVE, GENERATE, AND CRITIQUE THROUGH SELF-REFLECTION
Comparison
corrective_rag_2024.pdf
rag_lewis_2020.pdf
Research Problem
Large language models (LLMs) inevitably exhibit hallucinations, and retrieval
The paper addresses the challenge of improving pe
Dataset
PopQA dataset (used for fine-tuning the retrieval evaluator and evaluation);
The primary non-parametric knowledge source is a
Methodology
A plug-and-play framework (CRAG) that first evaluates the overall quality of
RAG models retrieve top-K text passages z for an in
Models/Algorithms
Retrieval evaluator fine-tuned on T5; baseline LLMs include LLaMA2, Alpaca
The architecture couples a dense passage retriever
Evaluation Metrics
Accuracy, FactScore, and Perplexity (as reported in Table 1 and Table 4).
Task-specific metrics are used: exact match / accur
Results
On the PopQA dataset, the T5-based retrieval evaluator achieved 84.3% acc
RAG achieves state-of-the-art results on Natural Qu
Limitations
The authors acknowledge that the retrieval evaluator's quality significantly d
The authors note that reliance on Wikipedia (or any
Table: reported by authors — pivoted from the structured analysis, not re-generated.
AI-GENERATED SYNTHESIS
Similarities
All three papers focus on retrieval-augmented generation (RAG) for knowledge-intensive NLP tasks, combining a retriever with a language model to improve factual accuracy. They each propose a plug-in or framework that extends standard RAG: CRAG adds corrective actions, SELF-RAG introduces self-critique and dynamic retrieval decisions, and the 2020 RAG paper establishes the baseline architecture. Each study evaluates on multiple datasets (PopQA, TriviaQA, etc.) and reports task-specific metrics such as accuracy, FactScore, or BLEU.
Differences


ResearchAgent
Agentic research assistant
LANGUAGE MODEL
Connected
Groq · openai/gpt-oss-20b
PAPERS
Upload
25MB per file • PDF
corrective_rag_2024...
16 pages · ok
rag_lewis_2020.pdf
19 pages · ok
self_rag_2023.pdf
30 pages · ok
AGENT
Retrieval & reasoning
Clear session
Decision support only — verify important claims against the original papers.
RESEARCH PAPER AGENT
ResearchAgent
Plans its own workflow · grounded in your papers · checks its own answers · literature-aware
LLM ready
Papers
3/3
ready / uploaded
Chunks
181
indexed for hybrid search
Questions
0
this session
Last run
—
no runs yet
Chat
Trace
Analysis
Report
Library
Security
About
Queries blocked
0
injection / misuse
Files rejected
0
type · encryption · active content
Injections caught
0
inside uploaded papers
Rate limit
0/120
0 per minute allowed
ACTIVE PROTECTIONS
Input guardrails
Prompt-injection and academic-misconduct requests are blocked before any LLM call. Unicode tricks (zero-width characters, full-width letters) are normalised first.
ML injection classifier
Llama Prompt Guard 2 scores every question; likely jailbreaks (score ≥ 0.8) are blocked, catching paraphrases the rules miss. Fails open with an audit event if unavailable.
File security
PDF magic bytes verified; encrypted PDFs and active content (/JavaScript, /Launch) rejected; duplicates detected by SHA-256; ≤25 MB, ≤80 pages, ≤5 papers.
Indirect prompt-injection defence
Instruction-like text inside papers is neutralised; evidence is wrapped in <evidence> tags and treated as data; fake delimiters are defused.
Grounding & verification


ResearchAgent
Agentic research assistant
LANGUAGE MODEL
Connected
Groq · openai/gpt-oss-20b
PAPERS
Upload
25MB per file • PDF
corrective_rag_2024...
16 pages · ok
rag_lewis_2020.pdf
19 pages · ok
self_rag_2023.pdf
30 pages · ok
AGENT
Retrieval & reasoning
Clear session
Decision support only — verify important claims against the original papers.
RESEARCH PAPER AGENT
ResearchAgent
Plans its own workflow · grounded in your papers · checks its own answers · literature-aware
LLM ready
Papers
3/3
ready / uploaded
Chunks
181
indexed for hybrid search
Questions
0
this session
Last run
—
no runs yet
Chat
Trace
Analysis
Report
Library
Security
About
corrective_rag_2024.pdf
rag_lewis_2020.pdf
self_rag_2023.pdf
Retrieval playground
See exactly what the RAG tool retrieves — compare keyword (BM25), dense and hybrid search.
Search query
e.g. evaluation metrics F1 accuracy
Mode
hybrid
dense
keyword

11. Limitations

- **Scanned PDFs** without a text layer can't be read, because there is no OCR; the agent reports this clearly instead of failing.
- **Tables, figures and equations** are extracted as plain text at best, so numbers that appear only in figures are missed.
- **Section detection is heuristic**: unusual heading styles fall back to the previous section label.
- **The verifier is itself an LLM**, so it reduces hallucinations but cannot guarantee their absence; the deterministic citation check covers only citation validity.
- **The prior-work check is shallow**: it shows the top 3 keyword matches from arXiv/OpenAlex, not a systematic literature review.
- **Scale**: the in-memory index is rebuilt every session and is designed for up to 5 papers; the Groq free tier's rate limits can slow large runs.
- **Injection defences reduce risk but can't guarantee detection.** The ML classifier (Prompt Guard 2) checks user questions only; document text relies on pattern neutralisation and delimiting.
- **Free-tier rate limits** (8k tokens per minute per model) make a first full 3-paper analysis take about 1–2 minutes. Later questions, and later visits by signed-in users, reuse the saved analyses.
- **Accounts are local** to the computer running the app, and sign-in lasts for the browser session. A public deployment would need HTTPS and persistent login cookies.
- **The benchmark is small** (15 questions on 3 papers). It catches regressions and shows trends, but it isn't a statistically strong measurement.

12. Future Scope

- OCR (e.g. Tesseract) for scanned papers, and table/figure extraction (e.g. layout-aware parsers).
- Smaller, overlapping chunks for dense method paragraphs, plus a cross-encoder re-ranker. These target the two remaining benchmark misses (Q10, Q12).
- Persistent vector store and per-user libraries, so papers are indexed once and reused.
- A cross-encoder re-ranker on top of hybrid retrieval; evaluation with RAGAS-style metrics (faithfulness, context precision).
- Semantic Scholar citation graphs to rank gaps by how much follow-up work exists; automatic BibTeX export.
- Run the Prompt Guard classifier over document chunks as well as user questions, and add authentication plus persistent audit storage for multi-user deployment.
- Multi-agent roles (reviewer agent, devil's-advocate agent) for richer critique of gaps.
- Human-in-the-loop plan editing: let the user approve or modify the plan before execution.